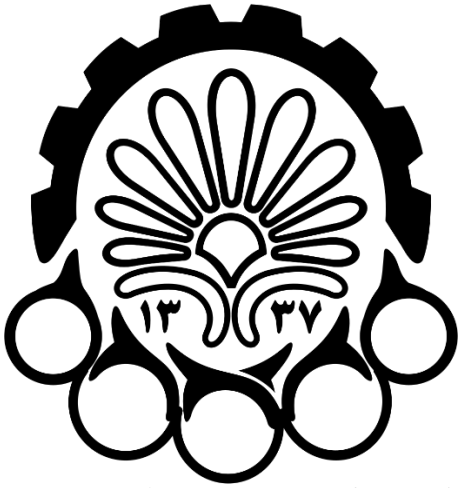


دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

گزارش پروژه سوم مبانی هوش مصنوعی

کیانا شرافتی ۴۰۱۳۱۰۲۲

توصیف محیط به‌صورت یک فرآیند تصمیم‌گیری مارکوف(MDP)

در گام نخست، محیط مورد مطالعه به‌صورت یک فرآیند تصمیم‌گیری مارکوف (MDP) مدل‌سازی می‌شود. هدف از این کار، ارائه‌ی یک چارچوب رسمی برای تحلیل رفتار عامل و ارزیابی سیاست‌ها در یک محیط کنترل‌شده است. برای این منظور، یک محیط ساده‌ی شبکه‌ای (Gridworld) با ابعاد 4×4 در نظر گرفته شده است که امکان تحلیل دقیق و شهودی رفتار عامل را فراهم می‌کند.

حالت‌ها(States)

محیط شامل ۱۶ حالت گسسته است که هر حالت متناظر با یک خانه از شبکه‌ی 4×4 می‌باشد. عامل در هر لحظه دقیقاً در یکی از این خانه‌ها قرار دارد. حالت‌ها به‌گونه‌ای شماره‌گذاری شده‌اند که موقعیت مکانی عامل در شبکه را نمایش می‌دهند. به این ترتیب، هر حالت نمایانگر یک موقعیت مکانی مشخص در محیط است.

اعمال ممکن

عامل در هر حالت (به‌جز حالت‌های پایانی) می‌تواند یکی از چهار عمل زیر را انتخاب کند:

- حرکت به سمت بالا
- حرکت به سمت راست
- حرکت به سمت پایین
- حرکت به سمت چپ

این اعمال به‌صورت قطعی تعریف شده‌اند؛ به این معنا که اگر عامل عملی را انتخاب کند، در صورت امکان، حتماً به خانه‌ی متناظر منتقل می‌شود. در صورتی که عامل بخواهد از مرزهای شبکه خارج شود، در همان خانه باقی می‌ماند.

پاداش‌ها(Rewards)

برای هر حرکت عامل، یک پاداش منفی ثابت در نظر گرفته شده است. این پاداش منفی را می‌توان به‌عنوان هزینه‌ی هر گام تفسیر کرد. در نتیجه، عامل به ازای هر حرکتی که انجام می‌دهد، مقداری جریمه دریافت می‌کند. این طراحی پاداش باعث می‌شود که عامل به‌طور طبیعی به دنبال مسیرهایی باشد که در کمترین تعداد گام به حالت‌های پایانی برسند.

حالت‌های پایانی(Terminal States)

در این محیط، دو حالت به‌عنوان حالت پایانی تعریف شده‌اند:

- خانه‌ی گوشه‌ی بالا-چپ شبکه
- خانه‌ی گوشه‌ی پایین-راست شبکه

زمانی که عامل به یکی از این حالت‌ها برسد، اپیزود به پایان می‌رسد و عامل دیگر حرکتی انجام نمی‌دهد. این حالت‌ها دارای ارزش صفر در نظر گرفته می‌شوند، زیرا پس از رسیدن به آن‌ها، هزینه‌ی اضافی به عامل تحمیل نمی‌شود.

تابع ارزش تحت سیاست تصادفی یکنواخت

پس از تعریف اجزای محیط، رفتار عامل تحت یک سیاست تصادفی یکنواخت مورد بررسی قرار می‌گیرد. در این سیاست، عامل در هر حالت غیرپایانی، بدون هیچ ترجیحی، هر یک از چهار عمل ممکن را با احتمال برابر انتخاب می‌کند. به بیان دیگر، عامل هیچ استراتژی خاصی برای رسیدن سریع‌تر به حالت‌های پایانی ندارد و تصمیمات آن کاملاً تصادفی است.

تابع ارزش محاسبه‌شده تحت این سیاست نشان می‌دهد که:

- حالت‌های پایانی دارای بیشترین ارزش (برابر با صفر) هستند.
- هرچه یک خانه از حالت‌های پایانی دورتر باشد، مقدار تابع ارزش آن منفی‌تر می‌شود.

تحلیل کیفی نتایج

این الگو به‌صورت شهودی کاملاً قابل توجیه است. از آنجا که عامل در هر حرکت جریمه دریافت می‌کند، ارزش هر حالت در واقع بیانگر میزان هزینه‌ی مورد انتظار برای رسیدن به یک حالت پایانی از آن موقعیت است.

خانه‌هایی که:

- فاصله‌ی کمتری تا یکی از حالت‌های پایانی دارند،
- به‌طور متوسط با تعداد گام کمتری می‌توانند به پایان اپیزود برسند،

دارای ارزش بالاتری (کمتر منفی) هستند.

در مقابل، خانه‌هایی که:

- در مرکز شبکه یا در نواحی دور از گوشه‌ها قرار دارند،
- انتظار می‌رود عامل برای خروج از آن‌ها تعداد بیشتری حرکت انجام دهد،

دارای ارزش‌های منفی‌تری هستند، زیرا عامل پیش از رسیدن به حالت پایانی، جریمه‌های بیشتری دریافت می‌کند.

به‌طور خلاصه، تفاوت ارزش خانه‌ها مستقیماً ناشی از فاصله‌ی آن‌ها تا حالت‌های پایانی و ماهیت هزینه‌محور پاداش‌ها است .سیاست تصادفی یکنواخت هیچ تلاشی برای کوتاه‌کردن مسیر انجام نمی‌دهد، بنابراین تابع ارزش حاصل، میانگینی از مسیرهای ممکن (کوتاه و بلند) را بازتاب می‌دهد.

تابع ارزش بهینه و سیاست بهینه حاصل ازPolicy Iteration

پس از محاسبه‌ی تابع ارزش تحت سیاست تصادفی یکنواخت، در گام بعدی با استفاده ا Policy Iteration سیاستی به دست می‌آوریم که به‌صورت سیستماتیک بهتر از سیاست تصادفی عمل می‌کند. Policy Iteration عملاً یک چرخه‌ی تکرارشونده است: ابتدا یک سیاست را ارزیابی می‌کند (می‌فهمد این سیاست در هر حالت "چقدر خوب است") و سپس با توجه به همان ارزیابی، سیاست را به شکل حریصانه بهبود می‌دهد؛ این روند ادامه پیدا می‌کند تا سیاست دیگر تغییر نکند.

در این محیط، هر حرکت یک هزینه‌ی ثابت منفی دارد. بنابراین اگر عامل بهینه رفتار کند، منطقی‌ترین کار این است که با کمترین تعداد گام خودش را به یکی از حالت‌های پایانی برساند. در نتیجه، مقدار ارزش هر خانه تقریباً برابر می‌شود با منفیِ حداقل تعداد قدم لازم تا نزدیک‌ترین ترمینال

این دقیقاً در جدول دیده می‌شود:

- خانه‌های ترمینال ارزش ۰ دارند.
- خانه‌های همسایه‌ی ترمینال ارزش ۱-دارند چون با یک حرکت می‌توانند به پایان برسند.
- هرچه یک خانه دورتر باشد، ارزشش به ترتیب ۲-، ۳-، ...می‌شود.

پس تابع ارزش بهینه یک ساختار بسیار منظم و هندسی دارد: مثل "نقشه‌ی فاصله" تا ترمینال‌ها.

سیاست بهینه

خروجی سیاست بهینه شما به صورت اندیس عمل‌ها بود (۰ بالا، ۱ راست، ۲ پایین، ۳ چپ). نکته‌ی مهم این است که این سیاست در هر خانه، عاملی را هدایت می‌کند که مسیر کوتاه‌تر به سمت یکی از دو گوشه‌ی پایانی را انتخاب کند.

اما چرا سیاست بهینه شما ممکن است در بعضی خانه‌ها "چند انتخاب" بهینه داشته باشد؟
چون در برخی موقعیت‌ها، فاصله‌ی عامل تا هر دو ترمینال یکسان است یا دو حرکت مختلف هر دو به یک

"کاهش فاصله‌ی یکسان" منجر می‌شوند. در چنین خانه‌هایی سیاست بهینه می‌تواند چند جواب درست داشته باشد، و پیاده‌سازی شما به‌خصوص اگر tie-breaking تصادفی نباشد ممکن است یکی از آن‌ها را ثابت انتخاب کند.

تفاوت سیاست بهینه با سیاست تصادفی: ساختار مسیرها

ساختار مسیرها در سیاست تصادفی یکنواخت

در سیاست تصادفی:

- عامل جهت مشخصی ندارد.
- ممکن است مدام بین خانه‌ها رفت و برگشت کند.
- ممکن است بارها به دیوار بخورد و در همان خانه بماند (چون حرکت به بیرون از شبکه باعث عدم جابجایی می‌شود).
- مسیرها معمولاً طولانی، پرنوسان و غیرمستقیم هستند.

نتیجه‌ی رفتاری:
عامل به طور متوسط گام‌های بسیار بیشتری تا رسیدن به ترمینال مصرف می‌کند، بنابراین هزینه‌ی تجمعی بیشتری می‌دهد و ارزش حالت‌ها شدیداً منفی می‌شود.

ساختار مسیرها در سیاست بهینه

در سیاست بهینه:

- مسیرها جهت‌دار و هدفمند هستند.
- عامل تقریباً همیشه حرکتی را انتخاب می‌کند که او را "یک خانه" به ترمینال نزدیک‌تر کند.
- مسیرها کوتاه، کم‌هزینه، و قابل پیش‌بینی می‌شوند.
- عامل از حرکت‌هایی که باعث درجا زدن یا دور شدن از هدف می‌شوند اجتناب می‌کند.

نتیجه‌ی رفتاری:
تعداد گام‌ها تا پایان به حداقل می‌رسد، بنابراین ارزش هر حالت فقط به "کمترین فاصله" وابسته می‌شود، نه به سرگردانی تصادفی.

تفاوت مقادیر ارزش: چرا ارزش‌ها در سیاست بهینه بالاترند؟

سیاست تصادفی

در سیاست تصادفی، ارزش خانه‌ها خیلی منفی‌تر است. دلیلش این است که:

- از یک خانه‌ی دور یا میانی، عامل ممکن است خیلی دیر به پایان برسد.
- چون حرکت‌ها تصادفی‌اند، زمان رسیدن به ترمینال یک متغیر تصادفی با امید ریاضی نسبتاً بزرگ است.
- هزینه‌ی هر حرکت جمع می‌شود، پس "امید هزینه" بزرگ و منفی می‌شود.

به زبان ساده:
خانه‌های وسط شبکه تحت سیاست تصادفی مثل یک باتلاق‌اند :عامل ممکن است خیلی طول بکشد تا از آن بیرون بیاید.

سیاست بهینه

در سیاست بهینه، ارزش‌ها خیلی منظم‌تر و کمتر منفی‌اند (بین ۰ تا ۳-). چون:

- عامل دقیقاً کوتاه‌ترین مسیر را می‌رود.
- بنابراین تعداد حرکت‌های لازم تقریباً قطعی و حداقلی است.
- هزینه‌ی کل هم حداقلی می‌شود.

به زبان ساده:
در سیاست بهینه هیچ سرگردانی وجود ندارد، بنابراین ارزش‌ها به شدت بهتر می‌شوند.

محیط LunarLander

فضای مشاهدات محیط شامل یک بردار هشت‌بعدی پیوسته است که وضعیت کامل فضاپیما را در هر لحظه توصیف می‌کند. این بردار شامل موقعیت افقی و عمودی فضاپیما، سرعت‌های افقی و عمودی، زاویه‌ی فضاپیما نسبت به سطح زمین، سرعت زاویه‌ای، و همچنین وضعیت تماس هر یک از پایه‌های فرود با زمین است. از آنجا که این متغیرها پیوسته هستند، نمی‌توان مستقیماً از آن‌ها در یک روش جدولی استفاده کرد.

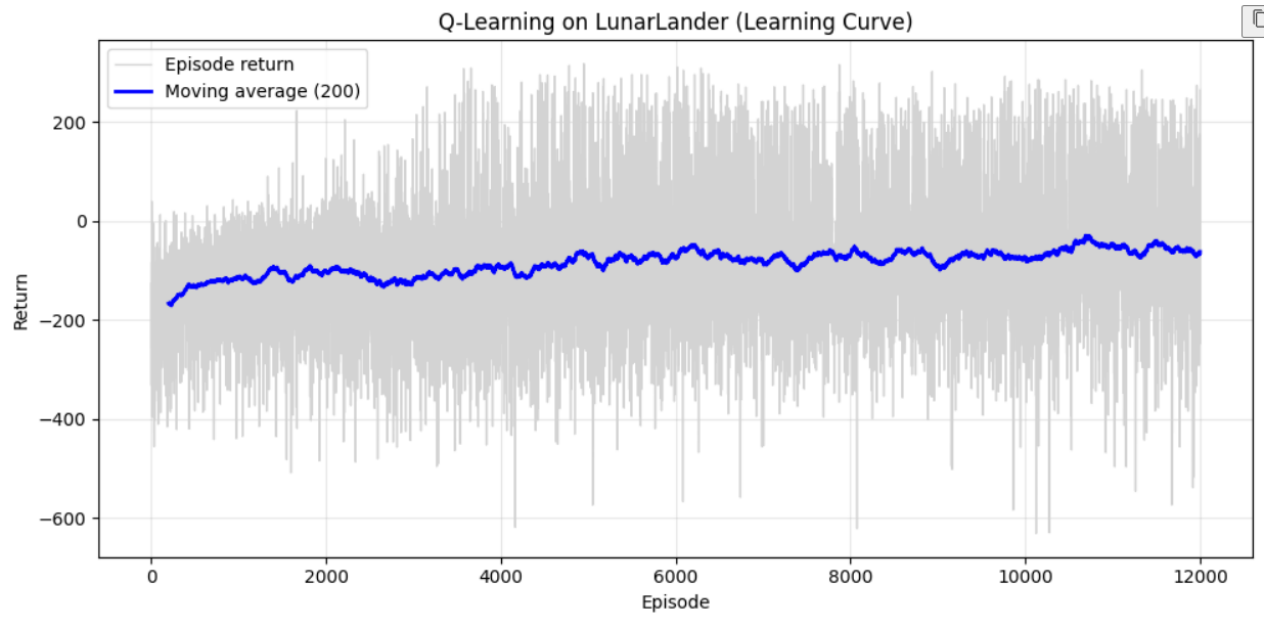
فضای اعمال محیط گسسته است و شامل چهار عمل ممکن می‌شود: خاموش بودن موتور، فعال‌سازی موتور اصلی برای کاهش سرعت سقوط، و فعال‌سازی هر یک از موتورهای جانبی برای کنترل تعادل و موقعیت افقی. عامل در هر گام زمانی یکی از این اعمال را انتخاب می‌کند.

برای اینکه بتوانم از الگوریتم Q-Learning جدولی استفاده کنم، حالت پیوسته‌ی محیط را گسسته‌سازی کردم. برای این کار، برای هر مؤلفه‌ی بردار مشاهده یک بازه‌ی معقول از مقادیر در نظر گرفتم و آن بازه را به چند بخش محدود تقسیم کردم. به این ترتیب، هر مشاهده‌ی پیوسته به مجموعه‌ای از اندیس‌ها نگاشت می‌شود که نشان می‌دهد مقدار هر ویژگی در کدام بازه قرار دارد. ترکیب این اندیس‌ها یک حالت گسسته را تشکیل می‌دهد که می‌توان از آن به‌عنوان ورودی جدول Q استفاده کرد.

پس از گسسته‌سازی حالت‌ها، الگوریتم Q-Learningرا پیاده‌سازی کردم. در این روش، عامل با استفاده از سیاست epsilon-greedy بین اکتشاف اعمال جدید و استفاده از بهترین اعمال یادگرفته‌شده تعادل برقرار می‌کند. جدول Q به‌صورت تدریجی و بر اساس تجربه‌ی عامل از تعامل با محیط به‌روزرسانی می‌شود؛ به این معنا که بعد از هر عمل، با توجه به پاداش دریافتی و وضعیت حالت بعدی، ارزش حالت–عمل فعلی اصلاح می‌شود.

در نهایت، این فرآیند باعث می‌شود عامل به‌تدریج سیاستی را یاد بگیرد که نسبت به سیاست تصادفی عملکرد بهتری دارد و بتواند فرودهای نرم‌تر و ایمن‌تری را در محیط LunarLander انجام دهد.

نمودار منحنی یادگیری براساس شماره اپیزود



مقایسه‌ی عددی بازده‌ها(Random vs Q-Learning)

ابتدا یک سیاست تصادفی را اجرا کردم که در هر گام به‌صورت کاملاً رندوم یکی از اعمال را انتخاب می‌کرد. خروجی این بخش به‌عنوان خط مبنا نشان می‌دهد بدون یادگیری، رفتار عامل معمولاً کنترل‌نشده است و بازده اپیزودها در اکثر مواقع پایین (اغلب منفی) باقی می‌ماند.

سپس عامل Q-Learning با همان محیط و با حالت‌های گسسته‌شده آموزش دادم. در مقایسه‌ی کلی، عامل یادگرفته‌شده نسبت به عامل تصادفی بازده بالاتری گرفت و روند کلی نشان می‌دهد در طول اپیزودها سیاست به سمت کنترل بهتر حرکت کرده است.

تحلیل کیفی رفتار بر اساس ویدیوهای ضبط‌شده

در بخش کیفی، رفتار دو عامل را با استفاده از ویدیوهای ضبط‌شده مقایسه کردم:

عامل تصادفی(Random policy)
در ویدیوی سیاست تصادفی تقریباً هیچ الگوی کنترل مشخصی دیده نمی‌شود. عامل یا موتور را در زمان‌های نامناسب روشن می‌کند یا اصلاً واکنش مؤثری به سرعت سقوط ندارد. نتیجه‌ی رایج این است که فضاپیما شدید سقوط می‌کند و با سر و وضعیتش آن‌قدر بی‌ثبات می‌شود که کنترل را از دست می‌دهد. به‌طور کلی، حرکت‌ها غیرقابل پیش‌بینی و بدون هدف هستند.

عامل یادگرفته‌شده(Q-Learning)
در ویدیوی عامل یادگرفته‌شده تفاوت واضح است: عامل معمولاً تلاش می‌کند سرعت سقوط را کمتر کند و رفتار آن نسبت به سیاست تصادفی کنترل‌شده‌تر است. در اجرای من، عامل اغلب به جای سقوط شدید، نرم‌تر سقوط می‌کرد و در بسیاری از اپیزودها برخورد نهایی با زمین به شکل قابل‌قبول‌تری اتفاق می‌افتاد یعنی با سر نبود.
با این حال، یک ضعف باقی‌مانده این بود که عامل دقیقاً در مرکز ناحیه‌ی هدف فرود نمی‌آمد و کمی بیرون‌تر از نقطه‌ی ایده‌آل می‌نشست یا در تنظیم دقیق موقعیت افقی ضعف داشت.

محدودیت‌ها

در این پروژه از Q-Learning جدولی برای محیط LunarLander استفاده شد. مهم‌ترین محدودیت این رویکرد، ماهیت پیوسته فضای حالت در LunarLander است. برای قابل‌استفاده‌بودن Q-Learning جدولی، ناچار شدم فضای مشاهدات را گسسته‌سازی کنم. این گسسته‌سازی باعث می‌شود بخشی از اطلاعات دقیق حالت از بین برود و عامل نتواند تفاوت‌های ظریف بین وضعیت‌های مشابه را به‌خوبی تشخیص دهد. در نتیجه، سیاست یادگرفته‌شده گاهی رفتار قابل‌قبولی دارد اما از نظر دقت و پایداری هنوز ایده‌آل نیست.

محدودیت دیگر، افزایش سریع اندازه‌ی جدول Q با زیاد شدن تعدادbin هاست. اگر بخواهیم دقت گسسته‌سازی را بالا ببریم، تعداد حالت‌ها به‌شدت زیاد می‌شود و هم حافظه‌ی بیشتری مصرف می‌کند و هم سرعت یادگیری را کاهش می‌دهد. بنابراین یک trade-off بین دقت نمایش حالت و عملی‌بودن الگوریتم وجود دارد.

همچنین، Q-Learningجدولی به‌شدت به استراتژی اکتشاف (exploration) وابسته است. اگر اکتشاف کافی نباشد، عامل ممکن است خیلی زود در سیاست‌های ضعیف گیر بیفتد، و اگر بیش از حد باشد، همگرایی سیاست نهایی کند یا ناپایدار می‌شود.

ایده‌های بهبود

برای بهبود نتایج می‌توان چند مسیر مشخص را پیشنهاد داد:

۱. گسسته‌سازی بهتر فضای حالت
به‌جایbin های یکنواخت، می‌توان از گسسته‌سازی غیر یکنواخت استفاده کرد؛ به‌طوری که در نواحی حساس‌تر (مثلاً نزدیک زمین یا نزدیک مرکز هدف) دقت بالاتری داشته باشیم. این کار کمک می‌کند عامل تصمیم‌های دقیق‌تری در مراحل پایانی فرود بگیرد.

۲. استراتژی‌های اکتشاف پیشرفته‌تر
به‌جای epsilon-greedy ساده، می‌توان از روش‌هایی مثل epsilon وابسته به حالت، یا کاهش تطبیقی epsilon بر اساس پیشرفت یادگیری استفاده کرد. این کار باعث می‌شود عامل در حالت‌های کمتر دیده‌شده بیشتر اکتشاف کند و در حالت‌های پرتکرار زودتر به بهره‌برداری برسد.

۳. تنظیم بهتر نرخ یادگیری
استفاده از نرخ یادگیری تطبیقی یا محدودکردن دامنه‌ی تغییرات Q-value می‌تواند نوسان‌ها را کاهش دهد و پایداری آموزش را افزایش دهد، به‌خصوص در مراحل پایانی یادگیری.

۴. حرکت به سمت روش‌های تقریبی
در نهایت، برای چنین محیطی که فضای حالت پیوسته و نسبتاً پیچیده دارد، استفاده از روش‌های مبتنی بر تقریب تابع مانند Deep Q-Network (DQN) می‌تواند بسیار مؤثرتر باشد. این روش‌ها بدون نیاز به گسسته‌سازی صریح، قادرند الگوهای پیچیده‌تر را یاد بگیرند و معمولاً عملکرد بهتری نسبت به Q-Learning جدولی دارند.